

Reviewer Pack — Minimal Rank of Free Entropy over Read-Twice BP Width

Entry 325aa4a1fe18 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 17:09:38 UTC

Minimal Rank of Free Entropy over Read-Twice BP Width

Entry ID: 325aa4a1fe18 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 17:09:38 UTC

1. Conjecture

Field A (mathematical branch): Information Theory (Free Entropy) **Field B** (complexity object): Complexity Theory: Read-Twice Branching Programs

Statement:

[‘For any read-twice branching program P with n inputs, the free entropy of P , denoted as $F(P)$, is upper-bounded by $\log_2(\text{size}(P)) + O(n)$.’]

Rationale (proposer’s reasoning):

[‘Free entropy has been used in information theory to capture the structure of probability distributions. By applying it to branching programs, we may uncover new structural properties that could distinguish between read-twice and unrestricted BP sizes.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5a627fa36e1f55c1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture that free entropy $F(P)$ of any read-twice BP P with n inputs is upper-bounded by $\log_2(\text{size}(P)) + O(n)$, a result will be considered supportive if, for all generated BPs up to size $n=40$, the mean difference between $F(P)$ and $\log_2(\text{size}(P)) + O(n)$ is less than or equal to 3 across at least 90% of seeds, and no seed’s $F(P)$ exceeds $\log_2(\text{size}(P)) + O(n)$ by more than 10 units.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "free entropy" AND "read-twice BP width" - "minimal rank" AND "free entropy" AND "branching program" - "complexity theory" AND "read-twice branching programs" AND "upper bound" free entropy

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=-9, elapsed=62.2s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def free_entropy(probabilities):
        return -sum(p * math.log2(p) for p in probabilities if p > 0)

    def generate_bp(n):
        size = 2 ** n
        bp = [random.random() for _ in range(size)]
        return bp

    def log_2_size(bp):
        return math.log2(len(bp))

    0_n = lambda n: random.uniform(0, 1) * n

    n_values = [5, 10, 15, 20, 30, 40]
    metric_values = []

    for n in n_values:
        bp = generate_bp(n)
        F_P = free_entropy(bp)
        upper_bound = log_2_size(bp) + 0_n(n)
        metric_values.append((F_P, upper_bound))

    mean_value = sum(F_P - upper_bound for F_P, upper_bound in metric_values) / len(metric_values)
    max_diff = max(abs(F_P - upper_bound) for F_P, upper_bound in metric_values)

    conjecture_holds = all(abs(mean_value) <= 3 and max_diff <= 10 for _ in range(9))
    counterexample = "" if conjecture_holds else "mean_diff={}, max_diff={}".format(mean_value, max_diff)
```

```

return {
    "metric_name": "free_entropy_bound",
    "metric_value": mean_value,
    "instances_tested": len(metric_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys

    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print("TRIAL: {}".format(result))
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_value, std_value, support_fraction))
    elif support_fraction >= 0.9:
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_value, std_value, support_fraction))
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print("RESULT: FALSIFIED counterexample=\"{}\" first_failing_seed={}".format(results[first_failing_seed], first_failing_seed))

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12692
2	propose	ollama_remote	glm4:latest	0	0	11115
3	propose	ollama_remote	glm4:latest	0	0	10254
4	preregistration	ollama_remote	glm4:latest	0	0	6270
5	novelty	ollama_remote	glm4:latest	0	0	4691
6	novelty	ollama_remote	glm4:latest	0	0	5566
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39642
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10084
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31738
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15569
11	judge	ollama_remote	glm4:latest	0	0	71631

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 219253 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/325aa4a1fe18.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/325aa4a1fe18.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/325aa4a1fe18.tar.gz` (if generated)